Software Engineering **TD 3**

Process Models + Risk Analysis + SE Principals **3RD YEAR - ENSIA**

Objectives:

- Introduction to Software Process Model
- Briefing students about their projects.
- Risk Analysis and SE principals.

INITIAL BRIEFING SESSIONS:

Each team will have a 5-minute meeting with the tutorial lecturer to discuss their project. During this session, the lecturer should check whether the team has an external client and confirm that they understand the client's needs and project requirements.

The goal of this briefing is to ensure that every team has a clear understanding of their project context, whether their client is internal or external, and that they are ready to begin preparing their Software Specification Document in the following weeks.

Task 1/Exercise C1 (Introductory):

Suggest the most appropriate generic software process model that might be used as a basis for managing the development of the following systems. Explain your answer according to the type of system being developed:

1. An interactive travel planning system that helps users plan journeys with the lowest environmental impact
2. A university accounting system that replaces an existing system
3. A system to control anti-lock braking in a car
4. Simple Calculator (Actual Hardware + Software)

Task2/Exercise C2 (Version Control and Configuration Management)

Scenario: A new junior developer joins your team. During their first week, they ask, "Why do we have to use a complex system like Git? I can just keep copies of my code in a folder on my computer named code_final_v2_my_changes.zip."

Task: Explain to this developer at least three distinct reasons why using a formal Version Control System (like Git) is essential for a professional software team, especially when compared to their manual folder-copying method.

Task3/Exercise 3: Risk Analysis and Mitigation (Spiral Model Application)

Objective: To simulate the first iteration of the Spiral Model, focusing on risk identification and the role of prototyping in reducing uncertainty.

Instructions: Your team is in the initial planning phase for a complex and ambitious project: A real-time language translation app that works on live video calls. This project is high-risk due to its technical complexity and performance requirements.

1- **Brainstorm Risks:** Identify at least five potential risks for this project. Classify each risk into one of the categories mentioned in the lecture (Technical, Schedule, Cost, Resource, Business, Security).

Example: (Technical) - The speech-to-text and translation APIs have too much latency, causing awkward delays in conversation.

2- **Prioritize Risks:** As a team, select the two most critical risks that could cause the entire project to fail.

3- **Propose a Mitigation Strategy**: For each of the two critical risks, describe a Prototype you could build to address that specific risk. Explain what the prototype would do and how it would help you resolve the uncertainty or prove the concept's viability before committing to full development.

Example (for latency risk): "We will build a simple 'proof-of-concept' prototype that connects two devices in a call. It will not have a user interface but will simply capture audio, send it to the translation API, receive the translation, and measure the round-trip time. This will tell us if the core technology is fast enough to be usable."

Task4/Exercise C4 (SE Management Principles) :

Fred Brooks, in his book "The Mythical Man-Month", outlined several core observations and principles about the challenges of managing complex software projects. Explain how your team would apply (or guard against) the following principles when developing the **"Quick Cup" Coffee Ordering App**. Provide concrete examples for each.

1. Conceptual Integrity

(The principle that a system should have a single, coherent, and consistent set of design ideas).

- **Question:** The core concept of the "Quick Cup" app is speed and simplicity. How would you ensure the app maintains this conceptual integrity? Give an example of a feature request the coffee shop owner might make that you would politely *reject* to preserve it.

2. The Second-System Effect

(The tendency for the second version of a system to be dangerously over-engineered because designers add all the features they skipped the first time).

- **Question:** Imagine your V1 of the app is the simple "Pay at Counter" version. For V2, the owner wants to add online payments, a loyalty points system, and push notifications for promotions. How would you specifically avoid the Second-System Effect while planning these new features?

3. The Mythical Man-Month
(The principle that "adding manpower to a late software project makes it later" due to increased communication and training overhead).

- **Question:** Your project is two weeks behind schedule. The coffee shop owner offers to have his nephew, who "is good with computers," join your team to "speed things up." Explain, using concrete examples of tasks (e.g., fixing the order queue bug, integrating the payment API), why adding this new person would likely delay the project even more.

Optional exercises to work on beyond class

Exercise P1 :

You are to develop a software product : discuss which one is better to offer as an interface to the end-user : GUI or CLI

Exercise P2 :

Briefly discuss why it is usually cheaper in the long run to use software engineering methods and techniques for software systems.

Exercise P3:

Software engineering is not only concerned with issues like system heterogeneity, business and social change, trust, and security, but also with ethical issues affecting the domain. Give some examples of ethical issues that have an impact on the software engineering domain.

Exercise P4 :

Discuss the three areas that need to be coordinated in a large software engineering project. Is any one of them more important than the others? Explain your conclusion.

Exercise P6 : What is meant by integration and why is it important to manage this effort for large systems?